PostgreSQL-Backed Transport Analytics for Paris and New York City

Maksym DOLHOV Mehdi AGHAEI Ho Bao Khanh Nguyen Nima DAVARI

April 16, 2026

Abstract

This project analyzes Paris and New York City public-transport demand through a PostgreSQL-first reporting pipeline. Source tables `public.idfm_daily_validations`, `public.idfm_hourly_profiles`, `public.mta_hourly_ridership`, `public.idfm`, and `public.mta` remain in the `public` schema, while the Python workflow reads reporting views in the `transport` schema: `v_daily_demand`, `v_nyc_hourly_profile`, `v_paris_hourly_profile`, `v_contributor_source`, and `v_city_structure_source`. The cleaned reporting scope includes NYC rows from 2020–2024 and Paris rows from 2015–2024, excludes the incomplete Paris 2025 year from comparison views, and restricts Paris contributor ranking to station-grain rows. Under this scope, the generated dataset contains 12,696 daily observations and 2,003 contributor-level rows. The workflow writes temporal summary files (`temporal_yearly_totals.csv`, `temporal_monthly_profile.csv`, `temporal_weekday_profile.csv`, `temporal_nyc_hourly_profile.csv`, `temporal_paris_hourly_profile.csv`), forecasting files (`forecast_metrics_overall.csv`, `forecast_metrics_by_city.csv`, `forecast_predictions.csv`), anomaly files (`anomaly_flags.csv`, `anomaly_rates.csv`), and contributor / structure files (`top_contributors.csv`, `city_structure_summary.csv`, and `city_monthly_structure.csv`) under `report/results/`.

1 Introduction

Urban transit systems generate dense operational traces that can support demand monitoring, comparative planning, and baseline forecasting. Smart-card and ridership logs preserve temporal structure at station or network level while remaining close to the decisions that transport operators need to make. For a student project, however, there is a second problem beyond the analysis itself: the workflow has to stay reproducible when multiple people work on notebooks, SQL, figures, and report text at the same time.

This project compares Paris and New York City transport demand through one shared analytical contract. Earlier repository iterations mixed local files, exploratory notebook material, and placeholder report claims. The final baseline removes that ambiguity. PostgreSQL stores the authoritative data, SQL views expose reporting-clean inputs, Python generates result tables and figures, and the report is written from those generated artifacts instead of from temporary notebook output.

The report analyzes Paris and NYC with four methods: temporal profiling, lag-based forecasting, anomaly detection, and contributor / city-structure analysis. The PostgreSQL views feed the notebook, result tables, figures, and report text from the same generated outputs. The analysis answers four questions: how the two systems differ over time, how well a lag-based baseline predicts daily demand, which observations are anomalous, and which stations account for the largest validated or observed totals.

2 Related Work

Smart-card data has long been used to recover travel behavior, rider segments, and network-level demand structure. Ma et al. [1] showed that transaction records can reveal stable travel patterns without relying on traditional survey frequency, while Long and Thill [2] combined smart-card data with household survey information to link ridership behavior with broader urban structure. Cats [3] further illustrated how smart-card traces can be mined for recurring mobility patterns at scale. These studies motivate the descriptive side of the current project, especially the use of station-level and city-level summaries instead of a single aggregate demand series.

Recent work also emphasizes richer behavior discovery. Aminpour and Saidi [4] used topic-modeling ideas to move beyond simple home-work simplifications, while Sun et al. [5] described how network analysis and smart-card records can expose structurally important stations. That literature is directly relevant to the contributor ranking used here: even when modeling remains simple, the rank order of highly used stations provides meaningful evidence about concentration, interchange hubs, and asymmetry between systems.

Forecasting literature is substantially more ambitious than the baseline used in this project. Dynamic graph and spatio-temporal learning approaches such as the model of Xie et al. [6] aim

to capture inter-station dependencies directly, while ASTIR [7] demonstrates how spatio-temporal data mining can support crowd-flow prediction. Those models are more expressive than the lag-based random forest baseline used here, but they also require more infrastructure and stronger assumptions about graph structure, feature availability, and tuning. The implemented baseline provides a transparent and reproducible comparison before heavier architectures are introduced.

Anomaly and disruption monitoring provide another important thread. Kolios et al. [8] studied monitoring recurrent mobility patterns through adaptive event-triggering logic, which is conceptually close to identifying observations that diverge strongly from routine behavior. The present project does not attempt causal attribution, but it uses anomaly scoring as an analytical screening layer that helps highlight periods worth interpretation in the report.

Other recent papers define tasks that are outside the scope of the current workflow. He et al. [9] analyzed factors that influence station ridership in a specific metro context, while Chen et al. [10] explored trip-purpose and socio-economic inference from transit users. Kundu et al. [11] examined heterogeneous effects from a new transit line. Those studies show how quickly transport analytics can become policy-rich and context-heavy. The current workflow uses that literature to define analytical questions, but it stops short of causal or socio-economic interpretation that the current dataset cannot support directly.

Taken together, the literature defines three tasks used in this paper: descriptive mining, baseline forecasting, and disruption-aware analysis. The implemented pipeline applies those tasks through a reporting-clean PostgreSQL contract, a Paris-vs-NYC comparison, and one generated result set from which the metrics, figures, and report text are derived. The scope is a comparative analytical workflow rather than a new predictive architecture.

3 Data Description

The project uses three operational transport datasets already loaded into PostgreSQL, together with two metadata tables. Paris demand is represented by `public.idfm_daily_validations`, which contains station-level daily validation counts, and `public.idfm_hourly_profiles`, which contains hourly distribution shares used for temporal shape analysis. NYC demand is represented by `public.mta_hourly_ridership`, an hourly ridership table that is later aggregated into comparable daily demand views. Reference tables `public.idfm` and `public.mta` provide descriptive network metadata such as station and network names.

The two cities are not perfectly symmetric in source structure. Paris provides direct daily station validations plus separate hourly profile shares, while NYC starts from hourly ridership observations that must be rolled up into daily station totals. The project therefore does not claim one-to-one measurement equivalence at every intermediate step. Instead, it defines a cleaned reporting scope where both systems can be compared at daily city and station level while preserving the source-specific hourly views needed for temporal interpretation.

Table 1: Primary data sources used in the paper

Source table	City	Temporal grain	Reporting coverage
IDFM daily validations	Paris	daily station validations	2015–2024
IDFM hourly profiles	Paris	hourly profile shares	cleaned reporting years
MTA hourly ridership	NYC	hourly station ridership	2020–2024

To ensure comparability, we exclude incomplete boundary years from the final comparison views; accordingly, the partial Paris 2025 observations are omitted from the reporting contract. Under this harmonized scope, the generated dataset contains 12,696 daily observations and 2,003 contributor-level station records, as reported in `analysis_summary.json`. This sample size is sufficient to support robust descriptive analysis and baseline predictive modeling, while remaining compact enough to preserve transparency, reproducibility, and interpretability across the workflow.

Two additional data choices are important. First, the contributor analysis uses station-grain Paris rows only, which avoids mixing line-level and station-level semantics in the ranking output. Second, the cleaned reporting views apply the same year-scope logic across daily, contributor, and city-structure outputs. This consistency matters because it prevents the paper from comparing annual summaries built from one scope with contributor totals built from another.

The generated result files reflect that same scope. Temporal outputs are written to separate yearly, monthly, weekday, and hourly CSV files; forecasting outputs are split into overall metrics, city metrics, and holdout predictions; anomaly outputs are split into row-level flags and city-level rates; and contributor outputs are separated from city structure summaries. This file-level separation makes it possible to trace each figure and table in the paper back to one concrete artifact rather than to a mixed notebook state.

Comparability therefore has to be handled procedurally rather than assumed. The paper does not argue that a Paris validation count is physically identical to an NYC ridership count at every level of aggregation. What it argues is narrower: once both systems are mapped into cleaned daily demand views with stable city labels, reporting scope, and contributor grain, they can support a defensible comparative analysis of trend, forecastability, anomaly concentration, and hub

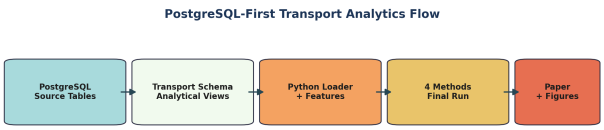

Figure 1: PostgreSQL-first workflow used for the generated outputs.

dominance.

4 PostgreSQL Workflow Architecture

PostgreSQL was chosen as the analytical backbone because the project needed one stable source of truth that multiple artifacts could consume without reloading or cleaning raw files separately. A CSV-first workflow would have left the report, notebook, and scripts vulnerable to local path drift and inconsistent intermediate cleaning steps. By contrast, PostgreSQL allows the raw uploaded tables to remain intact in the `public` schema while the cleaned reporting contract is expressed through SQL views in a separate `transport` schema.

That schema split is central to the design. The `public` schema holds source-level operational tables. The `transport` schema holds reporting-ready views such as `v_daily_demand`, `v_nyc_hourly_profile`, `v_paris_hourly_profile`, `v_contributor_source`, and `v_city_structure_source`. This means the SQL layer, not the notebook, defines the cleaned year scope and contributor label normalization used in the report. The Python workflow then consumes those views and writes the derived CSV outputs under `report/results/`, which are in turn used by the notebook, figure builder, and paper text.

This design improves reproducibility in two ways. First, it removes dependence on local raw files from the official workflow path. Second, it makes each quantitative claim traceable to either a SQL view or a generated result file. The workflow requirement for this report is one stable analytical contract; server deployment details are outside the paper's scope.

The contract is implemented as a multi-stage big data processing workflow. Initially, SQL views are used to normalize date ranges, standardize contributor identifiers, and compute daily aggregates. Subsequently, a Python pipeline extracts and engineers time-series features from these curated datasets and exports the processed outputs in CSV format.

Table 2: Key PostgreSQL analytical views

View	Role in the workflow
`v_daily_demand`	Cleaned daily demand baseline for downstream methods.
`v_nyc_hourly_profile`	NYC hourly ridership shape for temporal interpretation.
`v_paris_hourly_profile`	Paris hourly validation-share shape for temporal interpretation.
`v_contributor_source`	Station-grain contributor totals for ranking.
`v_city_structure_source`	City-level aggregates for structural comparison.

5 Methods

5.1 Temporal profiling

Temporal profiling is the descriptive baseline for the whole project. The daily demand view is summarized by year, month, and weekday to reveal long-horizon trends, seasonal structure, and weekly rhythm. The two hourly profile views are kept separate because Paris hourly data is based on validation shares, while NYC hourly data is based on observed ridership totals. This method produces the tables `temporal_yearly_totals.csv`, `temporal_monthly_profile.csv`, `temporal_weekday_profile.csv`, `temporal_nyc_hourly_profile.csv`, and `temporal_paris_hourly_profile.csv`. These files summarize the cleaned reporting layer by year, month, weekday, and hour. Because the method reads directly from the reporting views, implausible yearly totals or monthly patterns would indicate a problem in the SQL contract before later forecasting or anomaly outputs are interpreted.

5.2 Lag-based forecasting

The forecasting stage uses the cleaned daily demand series as input. The model is intentionally simple: a random-forest regressor trained on lag features, rolling-window statistics, and calendar attributes derived from the daily series. The numerical features are `lag_1`, `lag_7`, `lag_14`, rolling means and standard deviations over 7-day and 30-day windows, and one-step percentage change. Calendar features include day of week, week of year, day of year, month, year, and weekend status. Region, source, and metric type are one-hot encoded as categorical inputs. The split is time ordered, not random, so the model is evaluated on future observations relative to the training period. The random forest uses 220 trees with `min_samples_leaf=2`, which balances stability and transparency for the project scale. This produces `forecast_metrics_overall.csv`, `forecast_metrics_by_city.csv`, and `forecast_predictions.csv`. These files record holdout predictions and error metrics for the cleaned daily demand series under the lag-based baseline.

Table 3: Method-to-output mapping in the workflow

Method	Core inputs	Generated outputs
Temporal profiling	cleaned daily demand, hourly profile views	yearly, monthly, weekday, and hourly summaries
Forecasting	lag features, rolling features, calendar features	forecast metrics and holdout predictions
Anomaly detection	daily demand, lag features, rolling z-scores	anomaly flags and anomaly-rate summary
Contributors	station-level facts, city daily aggregates	top contributors and structural comparison tables

Table 4: Experimental setup artifacts

Artifact / source	Role	Grain / scope	Evidence
`v_daily_demand`	Shared daily baseline	city and station daily demand	yearly totals, forecast inputs
`v_nyc_hourly_profile`	Temporal shape analysis	hourly city profile views	hourly summaries
`v_paris_hourly_profile`	Temporal shape analysis	hourly city profile views	monthly summaries
`analysis_summary.json`	Result-size audit	12,696 daily rows; 2,003 contributor rows	workflow output summary
`forecast_metrics_overall.csv`	Holdout evaluation	cutoff 2023-01-03; 8,336 train / 4,276 test rows	forecasting protocol

5.3 Anomaly detection

The anomaly layer combines two signals: a rolling z-score that captures local deviation from recent behavior, and an `IsolationForest` model fitted on current value and lag-derived features. The anomaly feature set includes current value, 1-day and 7-day lags, a 7-day rolling mean and standard deviation, one-step percentage change, and a 30-day rolling z-score. The isolation model uses a contamination value of 0.03, while the statistical branch flags any observation with $|z| \geq 2.5$. This hybrid design helps separate routine daily variation from observations that are unusual both in local context and in the broader feature space. The output is written to `anomaly_flags.csv` and summarized in `anomaly_rates.csv`. The method is descriptive: it highlights unusual observations for interpretation, but it does not claim to identify the causal mechanism behind each anomaly.

5.4 Contributor and Structure Analysis

The method ranks stations by total demand contribution and summarizes each city through aggregate statistics such as average daily demand, variability, weekend sensitivity, and peak month. Paris contributor analysis is explicitly restricted to station-grain rows to avoid mixing station and line semantics in the ranking. NYC contributor totals are computed from the cleaned station daily demand layer. This stage produces `top_contributors.csv`, `city_structure_summary.csv`, and `city_monthly_structure.csv`. It complements the other methods by showing whether city-level differences are driven by broad network structure, dominant hubs, or both.

6 Experiments and Results

6.1 Experimental Setup

6.1.1 Execution environment

The database layer that backed the PostgreSQL query workflow ran on a Google Cloud `e2-standard-2` instance with 2 vCPUs, 8 GB of memory, an Intel Broadwell CPU platform, `x86/64` architecture, and no GPU. This hardware description is intentionally narrow: it documents the PostgreSQL host that served the analytical views used by the project, not the local machine used for Python execution, notebook inspection, or LaTeX compilation.

6.1.2 Datasets and reporting scope

The experiments use the same cleaned reporting contract described earlier. The core demand inputs are `public.idfm_daily_validations`, `public.idfm_hourly_profiles`, and `public.mta_hourly_ridership`; the reference tables are `public.idfm` and `public.mta`. The generated result set contains 12,696 daily observations and 2,003 contributor-level rows according to `analysis_summary.json`. Coverage in the cleaned yearly output spans 2020–2024 for NYC and 2015–2024 for Paris, with the incomplete Paris 2025 year excluded from the comparison-oriented views.

6.1.3 Protocols and evaluation

Each experiment uses one named protocol tied to generated artifacts. Temporal profiling aggregates the cleaned daily and hourly views into yearly, monthly, weekday, and hourly summaries. Forecasting uses a time-ordered holdout split with cutoff date `2023-01-03`, and reports MAE, RMSE, and MAPE from `forecast_metrics_overall.csv` and `forecast_metrics_by_city.csv`. Anomaly detection applies the hybrid rolling-z-score plus `IsolationForest` design and evaluates the output through anomaly-rate summaries rather than classification accuracy. Contributor and city-structure analysis ranks stations and summarizes city-level aggregates as a comparative descriptive experiment.

This evaluation logic matches the available data and task definitions. Forecasting is judged through holdout error because random shuffling would leak future information into training. City-level metrics are reported alongside overall metrics to avoid hiding

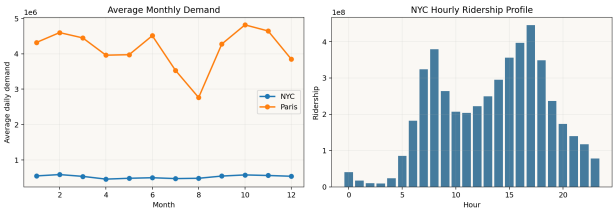

Figure 2: Current temporal outputs: monthly demand patterns and the NYC hourly profile.

Table 5: Selected yearly totals

City	2020	2023	2024
NYC	643.3M	1,163.3M	1,211.5M
Paris	926.0M	1,729.7M	1,024.2M

cross-city asymmetry inside one aggregate score. For anomaly detection, no ground-truth anomaly labels are available, so the report uses anomaly-rate summaries and qualitative interpretation of flagged periods. For contributor and city-structure analysis, plausibility of station rankings and consistency with temporal summaries act as the key validity checks.

6.2 Experiment 1: Temporal Profiling

The yearly totals confirm the effect of the reporting-clean scope. NYC covers 2020–2024, while Paris covers 2015–2024 with the incomplete 2025 year excluded. The Paris series shows a larger absolute scale throughout most of the window and a visible collapse in 2020 before a multi-year recovery. NYC shows a similar pandemic-era break, but the amplitude is smaller in absolute terms because the baseline level is lower.

This experiment uses the cleaned daily demand view together with the two hourly profile views. Its protocol is descriptive aggregation. The workflow produces yearly, monthly, weekday, and hourly-profile CSV summaries, and the paper reads those generated artifacts rather than issuing live analytical queries.

Monthly structure also differs. The generated monthly profile indicates that NYC peaks in February in the cleaned result set, while Paris peaks in October. Paris weakens sharply in August, which is consistent with seasonal travel behavior and reduced urban activity during summer holidays. At the weekly level, both systems show reduced weekend demand, but the later city-structure results confirm that the Paris weekend drop is stronger in relative terms.

The temporal outputs also show the forecasting context directly. The series are not stationary in a strict sense: both cities contain a pandemic-era structural break, strong seasonality, and pronounced weekly effects. The forecasting baseline is therefore evaluated on a series with visible structural breaks and recurring seasonal cycles rather than on a flat or trend-free process.

The selected yearly totals in Table 5 summarize the recovery pattern without requiring the full CSV output. Both systems rebound strongly after 2020, but the size and persistence of the recovery differ. NYC climbs steadily through 2024 in the cleaned result set, whereas Paris rebounds sharply by 2023 and then drops in the 2024 total present in the current reporting output. That pattern is part of the generated baseline and should be treated as an observed result in the current reporting scope.

Table 6: Forecast accuracy by city

City	Rows	MAE	RMSE	MAPE
NYC	3640	29,402	62,965	5.86%
Paris	636	187,446	376,361	6.94%

6.3 Experiment 2: Forecasting

According to `forecast_metrics_overall.csv`, the split trains on 8,336 rows and evaluates on 4,276 rows. The overall error reaches an MAE of 52,909, RMSE of 156,343, and MAPE of 6.02%. These values show that a simple lag-based baseline can already explain a large part of the daily structure in the cleaned reporting series.

This experiment uses the lag-based random-forest method defined earlier, but its protocol is explicit: the split is time ordered with cutoff date `2023-01-03`, the training set contains 8,336 rows, and the test set contains 4,276 rows.

The city-level split in Table 6 reveals an important asymmetry. NYC produces a lower MAE, lower RMSE, and lower MAPE than Paris. The most conservative reading is that the NYC series is easier to predict with the current feature set, while Paris contains stronger short-term swings or structure that the baseline model does not fully capture. The gap is not catastrophic: both cities still remain within a single-digit MAPE range. The city-level split therefore indicates that the NYC series is easier to predict with the current features, while the Paris series retains more variation that the baseline does not explain.

The absolute Paris error is much larger because the Paris city series operates at a higher traffic scale than NYC. For that reason, MAPE is a more meaningful cross-city comparison metric than MAE alone. Even under that normalized view, Paris remains harder to predict. This suggests that future work should either add richer exogenous context or move toward models that capture more complex temporal dependence across the network.

6.4 Experiment 3: Anomaly Detection

The anomaly outputs show the strongest contrast between the two systems. `anomaly_rates.csv` reports an anomaly rate of 0.73% for NYC and 8.96% for Paris. This difference is too large to ignore. It

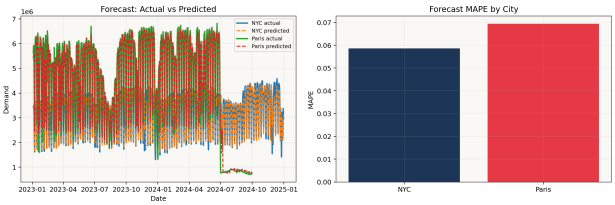

Figure 3: Forecast quality from the current result set.

Table 7: Anomaly rates by city

City	Source	Anomaly rate
NYC	mta_station_daily	0.73%
Paris	idfm_station_daily	8.96%

suggests that the Paris daily series contains many more observations that look unusual relative to their local and global context under the current anomaly design.

This experiment uses the hybrid anomaly protocol defined earlier: rolling z-scores capture local deviation and `IsolationForest` captures unusual points in the broader feature space. Because the project does not include ground-truth anomaly labels, the main evaluation artifact is `anomaly_rates.csv`, supported by row-level flags in `anomaly_flags.csv`.

The anomaly table does not by itself explain why those points are unusual, but the distribution is still meaningful. In Paris, major anomalies cluster around strong disruption and recovery periods, especially in the 2020 window. In NYC, the anomaly count is much lower and is concentrated around isolated weak-demand periods such as holidays or atypical weekends. This is precisely the type of screening result the method was designed to produce: it directs attention to time windows that deserve interpretation without claiming causal certainty.

The gap between the two rates is large enough that it should influence how the rest of the report is interpreted. A system with a higher anomaly rate is harder to summarize with one representative weekly pattern and is also more likely to challenge a simple forecasting baseline. In that sense, the anomaly section helps explain the forecasting asymmetry rather than functioning as an isolated appendix.

6.5 Experiment 4: Contributor and City-Structure Analysis

The city-level summary confirms that Paris is the larger system in the cleaned reporting scope. From `city_structure_summary.csv`, Paris reaches an average daily demand of approximately 4.13 million, while NYC reaches about 2.63 million. Median daily demand follows the same ordering, and both systems show similar coefficients of variation, around 0.43. The two systems therefore differ more in level and weekly sensitivity than in relative day-to-day volatility.

Table 8: City structure summary

City	Avg daily	Std. dev.	Wknd./wkday	Peak mo.
NYC	2.63M	1.15M	0.584	2
Paris	4.13M	1.76M	0.512	10

Table 9: Illustrative top contributors

City	Station	Total demand
Paris	SAINT-LAZARE	432.3M
Paris	LA DEFENSE-GRANDE ARCHE	325.3M
Paris	GARE DE LYON	291.6M
NYC	Times Sq–42 St	168.2M
NYC	Grand Central–42 St	115.4M
NYC	34 St–Herald Sq	97.4M

Source: `top_contributors.csv`.

This experiment is a comparative descriptive protocol. It reads `city_structure_summary.csv`, `city_monthly_structure.csv`, and `top_contributors.csv` to measure level, variability, weekend sensitivity, peak month, and ranked station contribution without introducing a separate predictive model.

The weekend-to-weekday ratio makes the weekly difference concrete. NYC keeps a higher weekend share (0.584) than Paris (0.512), which implies that Paris demand is more concentrated in weekday commuting structure. This is consistent with the seasonal and temporal differences reported earlier.

The contributor ranking is also plausible after the Stage 2 label cleanup. Paris is dominated by Saint-Lazare, La Défense–Grande Arche, Gare de Lyon, Gare du Nord, and Montparnasse, all of which are obvious interchange or high-throughput nodes. NYC is led by Times Sq–42 St, Grand Central–42 St, 34 St–Herald Sq, and 14 St–Union Sq, which again aligns with domain expectations. The rank order is consistent with major interchange hubs in both systems, which indicates that the cleaned labels and contributor aggregation are producing plausible dominant stations.

The top-contributor view also clarifies why city-level comparisons should not be read as if the two systems were structurally identical. Paris is more top heavy in absolute station totals, while NYC spreads more of its total demand across a larger set of high-volume complexes. That does not invalidate the city comparison; it simply means the comparison is most defensible at the level of cleaned daily totals, weekend sensitivity, and ranked hubs rather than at raw network topology.

6.6 Cross-method synthesis

The four methods are more informative when read together than in isolation. Temporal profiling shows that both cities have strong weekly and seasonal structure, but Paris carries the larger scale and the sharper summer weakening. Forecasting then shows

Table 10: Key quantitative findings

Question	Evidence
Which city has higher average daily demand?	Paris, about 4.13M versus 2.63M for NYC.
Which city is easier to forecast?	NYC, with 5.86% MAPE versus 6.94% for Paris.
Which city shows more unusual days?	Paris, with an 8.96% anomaly rate versus 0.73% for NYC.
What dominates station demand?	Major interchange hubs, led by Saint-Lazare and Times Sq–42 St.

Sources: `city_structure_summary.csv`, `forecast_metrics_by_city.csv`, `anomaly_rates.csv`, and `top_contributors.csv`.

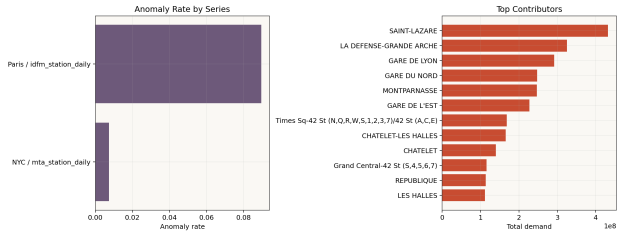

Figure 4: Anomaly-rate contrast and top contributors in the cleaned result set.

that this larger and less regular Paris signal is harder to predict with the same lag-based baseline. The anomaly layer reinforces that reading by flagging a much higher share of unusual Paris observations. Finally, the contributor ranking shows that Paris demand is more concentrated in a set of very large hubs, while NYC distributes high demand across several major interchange complexes.

The daily summaries, anomaly tables, and contributor rankings all come from one cleaned view layer. As a result, these sections use one city scope, one contributor grain, and one set of generated outputs.

7 Discussion and Limitations

The workflow defines a reproducible analytical baseline, not a claim that the present method stack is the best available for transport demand modeling. The forecasting component remains a baseline model. It is transparent, quick to rerun, and suitable for comparing the two cleaned city series, but it does not model richer network or event effects in the way graph-based and more specialized temporal models might [6, 7].

The anomaly layer should also be interpreted carefully. A flagged observation is evidence of unusual behavior, not proof of a particular operational cause. In this report, anomaly detection functions as a triage layer that marks periods for human interpretation or future event matching.

Contributor analysis remains sensitive to source labeling. The project removed mechanical whitespace

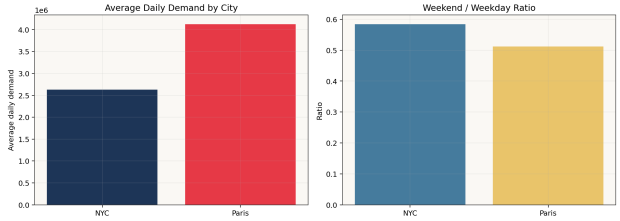

Figure 5: Average daily demand and weekend sensitivity by city.

inconsistencies and restricted Paris ranking to station rows, but semantically related stations can still appear separately when their source identifiers differ. That is acceptable for the current paper because the top-ranked stations remain plausible, but it limits how aggressively the report should interpret small differences among nearby hubs.

Finally, the reproducibility contract depends on generated outputs rather than live database queries at paper compile time. This is a reasonable compromise for a project report: the SQL views and workflow define one stable path, and the paper cites the outputs of that path. Still, if the database changes, the workflow must be rerun before the report should be updated.

Compared with the earlier repository state, the notebook, result tables, figures, and report now read from regenerated PostgreSQL outputs rather than from separate narratives, manually assembled values, or placeholders.

The repository history shows what happens when data sources, notebook text, SQL logic, and report claims drift apart: individual metrics become difficult to trace or defend. The PostgreSQL-first rewrite reduces that risk by keeping those artifacts on one result contract.

At the same time, the report should not hide the remaining frontier. Better forecasting likely requires richer external context, more explicit modeling of network dependence, or both. Better anomaly interpretation would require event labeling or an external disruption calendar. More aggressive contributor harmonization would require domain judgment about semantically related stations. Those are meaningful next steps, but they are extensions to a stable baseline, not fixes for a broken workflow.

For planning-oriented interpretation, the current outputs indicate that a city with stronger weekday concentration and higher anomaly rate requires more caution when extrapolating routine patterns into future demand. A system whose dominant stations absorb a large share of total volume may also be more sensitive to local disruptions at those hubs. These statements remain descriptive rather than causal because the project does not include external event labels or intervention data.

8 Conclusion

This project delivers a coherent analytical baseline for comparing Paris and NYC transport demand. PostgreSQL serves as the single source of truth, reporting-clean views in the `transport` schema define the shared contract, and the Python workflow generates the figures and CSV outputs used by the notebook and paper. Within that framework, four methods were enough to produce interpretable findings: temporal profiling established the seasonal and weekly structure, the lag-based forecasting baseline achieved single-digit MAPE in both cities, anomaly detection highlighted much stronger irregularity in Paris, and contributor analysis confirmed plausible dominant stations in both systems.

9 AI Use

Large language models were used as editing support to help refactor and restructure parts of the codebase and report text during the project. Their use was limited to workflow and writing assistance rather than as a source of final project evidence. All quantitative claims, generated outputs, and final report statements were checked against the repository artifacts and generated result files before inclusion in the report.

References

[1] X. Ma, Y.-J. Wu, Y. Wang, F. Chen, and J. Liu, "Mining smart card data for transit riders' travel patterns," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 1–12, 2013.

[2] Y. Long and J.-C. Thill, "Combining smart card data and household travel survey to analyze jobs–housing relationships in Beijing," *Computers, Environment and Urban Systems*, vol. 53, pp. 19–35, 2015.

[3] O. Cats, "Identifying human mobility patterns using smart-card data," arXiv preprint arXiv:2208.05352, 2022.

[4] N. Aminpour and S. Saidi, "Unveiling mobility patterns beyond home/work activities: A topic modeling approach using transit smart card and land-use data," *Sustainable Cities and Society*, vol. 114, p. 105745, 2024.

[5] L. Sun *et al.*, "Optimizing urban mobility through complex network analysis and big data from smart cards," *IoT*, vol. 6, no. 3, p. 44, 2025.

[6] P. Xie, M. Ma, T. Li, S. Ji, S. Du, Z. Yu, and J. Zhang, "Spatio-temporal dynamic graph relation learning for urban metro flow prediction," arXiv preprint arXiv:2204.02650, 2022.

[7] M. Lablack, H. Qi, Y. Shen, and B. Yin, "ASTIR: Spatio-temporal data mining for crowd flow prediction," *IEEE Access*, vol. 7, pp. 175159–175165, 2019.

[8] P. Kolios *et al.*, "Model-adaptive event triggering for monitoring recurrent mobility patterns in public transport," *IEEE Transactions on Intelligent Transportation Systems*, 2023.

[9] Y. He, Y. Zhao, and K. L. Tsui, "An analysis of factors influencing metro station ridership: Insights from Taipei Metro," in *Proc. IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 1598–1603.

[10] Y. Chen *et al.*, "A two-stage trip inference model of purposes and socio-economic attributes of regular public transit users," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[11] D. Kundu *et al.*, "Benefits from a new transit line: Exploring the impact of rare users and spatially heterogeneous variations in intensity of use," *Journal of Public Transportation*, vol. 27, p. 100120, 2025.